

Appendix B: certificate architecture

Repository source: proof/APPENDIX_B_CERTIFICATE_ARCHITECTURE.md

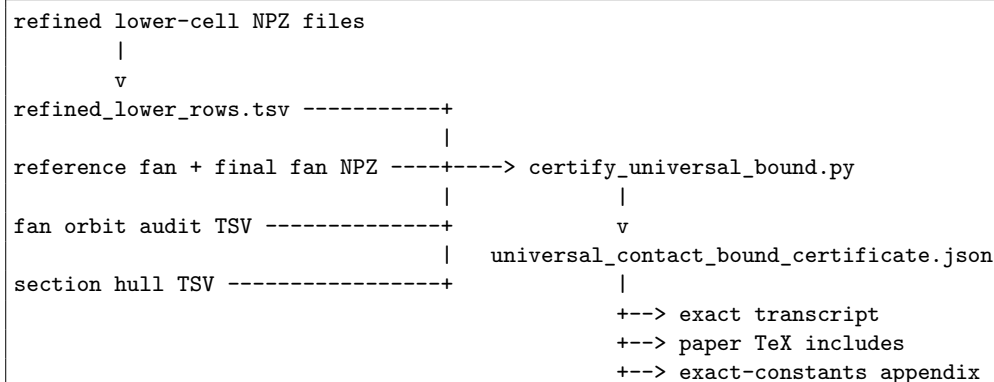
Contents

B.1 Object graph	1
B.2 Authoritative inputs	1
Lower branch	1
Near-Jung fan	2
Sections and cap allocation	2
Assembly certificates	2
B.3 Exact programs	2
B.4 Derived files	2
B.5 Independent implementations	3
B.6 Regression and adversarial tests	3
B.7 Fingerprints and manifests	3
B.8 Trust and failure boundaries	3
B.9 Versioning rule	3

Status: human-readable map of the exact package **Proof authority:** the exact verifier and immutable machine-readable objects

B.1 Object graph

The public package has one self-contained dependency graph:



All paths resolve inside this repository. No private parent directory, sibling checkout, network service, or hidden cache is required.

B.2 Authoritative inputs

Lower branch

```
certificate/certificates/refined_lower_cell_1.npz
certificate/certificates/refined_lower_cell_2.npz
certificate/certificates/refined_lower_cell_3.npz
certificate/certificates/refined_lower_cell_4.npz
```

```
certificate/certificates/refined_lower_rows.tsv
```

The NPZ files contain exact integer grid data for the four replacement cells. The TSV file records the complete gap-free row partition and exact coefficient-of- π comparisons.

Near-Jung fan

```
certificate/certificates/reference_near_jung_fan.npz
certificate/certificates/universal_contact_fan.npz
certificate/certificates/fan_orbit_margin_audit.tsv
```

The reference fan anchors the inherited geometry. The final fan stores exact integer radial numerators, exact deficiency numerators, the dyadic scale, and volume data. The orbit audit records a canonical representative, orbit size, active bound, multiplier sign data, and margin rendering for every stabilizer orbit.

Sections and cap allocation

```
certificate/certificates/all_axis_pair_section_hulls.tsv
certificate/certificates/audit_constants.csv
```

The section-hull file gives exact rational polygon vertices for all transformed axis/sign pairs. The constants file is an audit-oriented decimal/rational view of selected derived quantities.

Assembly certificates

```
certificate/certificates/reference_handoff_certificate.json
certificate/certificates/universal_contact_bound_certificate.json
```

The reference handoff object preserves the predecessor interface used for standalone export equivalence. The universal certificate is the final assembled machine-readable theorem object.

B.3 Exact programs

```
certificate/exact_geometry.py
certificate/certify_universal_bound.py
```

`exact_geometry.py` contains reusable exact geometry and rational helpers. `certify_universal_bound.py` performs the production verification and byte-identical reconstruction.

The proof path uses Python integers and `fractions.Fraction`. NumPy is used to read the immutable NPZ integer arrays with `allow_pickle=False`; it is not used for theorem-bearing floating-point comparisons.

B.4 Derived files

These files are deterministic projections of the authority graph:

```
certificate/certificates/universal_contact_bound_certificate.json
certificate/transcripts/exact_certificate.txt
paper/certificate_constants.tex
paper/lower_branch_table.tex
proof/APPENDIX_A_EXACT_CONSTANTS.md
```

The JSON and exact transcript are rebuilt by the exact verifier. The paper includes are built or checked by `paper/reconcile_certificate.py`. Appendix A is built or checked by `proof/generate_exact_constants.py`.

A stale or nonidentical derived file causes the verification path to stop.

B.5 Independent implementations

```
certificate/audit_universal_bound.py
certificate/audit_universal_bound.R
```

The Python audit reconstructs the proof numerically in binary64. The R audit uses base R and independently parses archived values and rebuilds the cap and final assembly. Neither is proof authority. Their purpose is to make common implementation, parsing, and transcription errors visible.

B.6 Regression and adversarial tests

```
certificate/test_universal_bound.py
certificate/test_adversarial_mutations.py
```

The ten reconstruction tests cover the certificate claims, final fan, lower-row coverage, determinant census, coordinate widths, cap assembly, reference fingerprints, exact rebuild, binary64 audit, and standalone paths.

The seven mutation tests deliberately corrupt one object at a time:

1. lower-row endpoint;
2. fan radial numerator;
3. S-procedure multiplier;
4. section-hull vertex;
5. cap bracket;
6. disjointness threshold;
7. theorem coefficient digit.

All seven must be rejected.

B.7 Fingerprints and manifests

The package uses layered SHA-256 manifests:

```
SHA256SUMS.txt
certificate/SHA256SUMS.txt
paper/SHA256SUMS.txt
rendered/markdown/SHA256SUMS.txt
```

The root manifest authenticates the complete release-facing tree except explicitly ignored build and environment directories. The nested manifests make the certificate, paper, and rendered documentation independently easy to check.

A checksum proves file identity, not mathematical correctness. Semantic exact checks remain necessary even when all hashes match.

B.8 Trust and failure boundaries

The verifier can establish that the checked finite objects imply the encoded coefficient under the mathematical reductions implemented in the package. It cannot by itself establish that:

- the manuscript’s geometric reduction has no omitted case;
- Python’s integer and `Fraction` implementations are flawless;
- the operating system or hardware is nonmalicious;
- the certificate is novel;
- the theorem has been independently reviewed.

Those boundaries are documented in [certificate/TRUST_BOUNDARY.md](#).

B.9 Versioning rule

The following require a new certificate version and explicit mathematical review:

- any change to a proof-object value;

- any change to exact verification logic;
- any change to the theorem numerator or denominator;
- any change to the certified radius partition;
- any change that weakens or removes a semantic rejection test.

Editorial prose changes, PDF rebuilds, and link repairs do not change the mathematical theorem, but they still require refreshed checksums and release notes.